

- Background Knowledge & Technical Q&A
 - 1. Background Knowledge
 - PostgreSQL basics
 - Indexes
 - EXPLAIN ANALYZE
 - qPCR / assay context (minimal)
 - Streamlit / app architecture
 - 2. Potential Technical Questions and Answers

Background Knowledge & Technical Q&A

Use this document to review concepts before the interview and to prepare answers for the 10–15 minute technical Q&A.

1. Background Knowledge

PostgreSQL basics

- **Tables, primary keys (PK), foreign keys (FK)**

A PK uniquely identifies a row. A FK in table B references a PK (or unique column) in table A, so every value in B must exist in A. FKs enforce referential integrity: you cannot have a run pointing to a non-existent assay, and you can optionally CASCADE deletes (e.g. deleting a run deletes its run_sample and qc_result rows).

- **UNIQUE constraints**

One value (or combination of columns) can appear at most once in the table. We use UNIQUE on sample.external_id, (assay_id, run_name) on assay_run, and (run_id, sample_id, replicate_number) on run_sample.

- **CHECK constraints**

Restrict values at insert/update time. Examples: status IN ('COMPLETED', 'FAILED', ...), matrix_type IN ('Plasma', 'Serum', ...), well_position matching regex for A01–H12, result_flag IN ('PASS', 'WARN', 'FAIL').

- **Partial unique indexes**

Enforce uniqueness only on a subset of rows. We use two partial uniques on qc_result: one on (run_sample_id, qc_rule_id) WHERE run_sample_id IS NOT NULL (sample-level QC), and one on (run_id, qc_rule_id) WHERE run_sample_id IS NULL (run-level QC).

Indexes

- **Purpose**

Speed up WHERE, JOIN, and ORDER BY by avoiding full table (sequential) scans. The planner chooses when to use an index based on estimated cost.

- **Composite index column order**

Put equality filters first, then range or ORDER BY columns. Example: (status, started_at DESC) on assay_run supports “WHERE status = 'COMPLETED' ORDER BY started_at DESC”.

- **Partial index**

Only indexes rows matching a WHERE clause (e.g. result_flag = 'FAIL'). Smaller index, faster for queries that filter on that condition.

EXPLAIN ANALYZE

- **What it shows**

The actual execution plan: which tables are scanned, in what order, using index or sequential scan, and actual row counts and execution time.

- **Index Scan vs Seq Scan**

Index Scan = use index to find rows (good for selective filters). Seq Scan = read the whole table (can be correct when most rows are needed or when the table is small).

- **What to look at**

“actual time” and “rows” (and “loops”) matter more than “cost” for interpretation. “Buffers: shared hit” means data was read from cache (efficient).

qPCR / assay context (minimal)

- **Run** — One execution of an assay on an instrument (e.g. one 96-well plate). Has status: SCHEDULED, RUNNING, COMPLETED, FAILED.
- **Sample** — A biological sample; can appear in many runs (replicates, repeat testing). Identified by `external_id` (e.g. barcode from LIMS).
- **QC** — Quality control rules (e.g. Ct in range 15–35, NTC Ct > 35). Each rule produces a result: PASS, WARN, or FAIL.
- **Ct** — Cycle threshold from qPCR. **NTC** — No-template control (should not amplify).
- **Well** — Position on the plate (A01–H12 for 96-well).

Streamlit / app architecture

- **Reruns** — On each interaction, Streamlit reruns the script. Use `st.cache_resource` so the DB connection is created once and reused.
 - **Thin UI** — Sidebar filters → call functions in `app/queries.py` with parameters → display returned DataFrames (tables/charts). No business logic or SQL in the page modules; SQL lives in `app/queries.py`.
-

2. Potential Technical Questions and Answers

Q1: Why did you use a bridge table (`run_sample`) instead of putting `run_id` on `sample`?

A sample can be in many runs (replicates, repeat runs); a run has many samples. That's a many-to-many relationship, so we need a bridge table. `run_sample` holds `run_id`, `sample_id`, `replicate_number`, `well_position`, `expected_ct`, and status for each “sample-in-a-run” occurrence.

Q2: Why is `external_id` unique on `sample`?

`external_id` represents the external barcode or LIMS identifier. We need a stable, unique key to look up samples and trace them across runs (e.g. “show me all QC history for sample SMP-0001”). The unique constraint also prevents duplicate sample records for the same barcode.

Q3: Why partial unique indexes on qc_result?

Sample-level QC: one row per (run_sample_id, qc_rule_id). Run-level QC: one row per (run_id, qc_rule_id). We use two partial unique indexes—one WHERE run_sample_id IS NOT NULL and one WHERE run_sample_id IS NULL—so we enforce the right uniqueness for each case without mixing them in a single constraint.

Q4: Why CHECK on well_position?

To enforce the 96-well plate layout (A01–H12). Invalid positions are rejected at insert time. The CHECK uses a regex: `^A-H$`.

Q5: How did you decide which indexes to create?

I identified three main query patterns (run summary, run drill-down, sample history), looked at the WHERE, JOIN, and GROUP BY columns in each, and added indexes that match those access paths—e.g. run_id on run_sample, run_sample_id on qc_result, external_id on sample. I ran EXPLAIN ANALYZE to confirm the planner uses these indexes.

Q6: What does EXPLAIN ANALYZE show for the drill-down query?

Index Scan on idx_run_sample_run (one run's wells, e.g. 96 rows), then Index Scan on idx_qc_result_run_sample for each well to fetch QC. Execution Time is around 1.7 ms. So both indexes are used and the query is very fast.

Q7: Why is there a Seq Scan in the run summary plan?

For the global run summary we join and aggregate over many runs and all their run_sample and qc_result rows. At current data size the planner chose to sequential scan qc_result and run_sample and use hash joins. That's acceptable for this scale; for much larger data we could consider partitioning (e.g. by run_id or time) or additional indexes.

Q8: Why Streamlit instead of a full front-end framework?

The goal was a minimal, interview-ready demo focused on schema, indexing, and SQL. Streamlit provides a quick UI (filters, tables, charts) without writing HTML/CSS/JS or building a separate API. The focus stays on database design and performance.

Q9: How do you avoid SQL injection?

All user inputs (dates, run_id, external_id, status lists) are passed as parameters to the query (e.g. %s in psycopg), not concatenated into the SQL string. See app/queries.py:

params are built in a list and passed to execute().

Q10: Why store both metric_value and result_flag in qc_result?

metric_value is the raw measurement (e.g. Ct value); result_flag is the outcome of the rule (PASS/WARN/FAIL). Storing both supports auditing (“what was the actual Ct?”) and re-evaluation if thresholds change later.

Q11: What would you add if this went to production?

Examples: authentication and authorization, audit logging (who changed what), connection pooling (e.g. PgBouncer), rate limiting, input validation in the app layer, backups and point-in-time recovery, and for very large scale possibly partitioning qc_result by time or run_id.

Q12: Why ON DELETE CASCADE on assay_run and run_sample?

When a run is deleted, its run_sample rows and their qc_result rows have no meaning without the run. CASCADE keeps the database consistent without orphan rows. We do not CASCADE on sample deletion so that sample history is preserved or we can handle it explicitly.

Q13: What is the role of qc_rule?

qc_rule defines the rules (name, applies_to RUN/SAMPLE, metric_name, operator, thresholds). qc_result references qc_rule so each result is tied to the rule that was evaluated. This allows adding or changing rules without changing the schema and keeps rule definitions in one place.

Q14: Why LEFT JOIN to qc_result in the three main queries?

Some run_sample rows may have no QC results yet (e.g. run in progress), or we want to list all wells even if a few have no QC. LEFT JOIN keeps all run_sample (or sample history) rows and shows NULL for missing QC.

Q15: How does the app get its database connection?

app/config.py reads DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD from the environment. app/db.py uses st.cache_resource so get_connection() is called once and the same connection is reused across Streamlit reruns.

Q16: What if two runs have the same run_name?

They cannot, for a given assay. The constraint `UNIQUE(assay_id, run_name)` on `assay_run` enforces that. So “RUN-N1-001” can exist once per assay, which matches the business rule that run names are unique per assay.

Q17: Why not use an ORM (e.g. SQLAlchemy)?

For this demo we wanted the SQL to be explicit and visible—easier to discuss schema, indexing, and `EXPLAIN` in an interview. An ORM would hide the actual queries and add abstraction without benefit for a small, query-centric app.

Q18: What is the partial index `idx_qc_result_failed_only` used for?

It indexes only rows where `result_flag = 'FAIL'`. Queries that filter or aggregate only failed QC results (e.g. “list all failed samples in this run”) can use this smaller index instead of scanning all `qc_result` rows.

Q19: How do you run `EXPLAIN ANALYZE` for the demo?

`scripts/run_explain.py` connects using `app/config.py`, runs `EXPLAIN (ANALYZE, BUFFERS, VERBOSE)` for the three key queries with representative parameters, and writes the output to `explain/run_summary.txt`, `explain/run_drilldown.txt`, and `explain/sample_history.txt`. The Streamlit “Advanced” expanders display these files when present.

Q20: Why `BIGSERIAL` for primary keys?

`BIGSERIAL` gives 64-bit auto-incrementing integers, so we have room for very large tables (millions of runs, samples, QC results) without running out of key space. For a demo it’s slightly more than needed but is a safe, common choice.